

LOCAL ENCRYPTED STORAGE

Từ một file bị đánh cắp đến một thiết kế mật mã có thể kiểm chứng

Một bài toán. Một attacker. Một contract. Không học lan man.

SAU BÀI NÀY, BẠN SẼ HIỂU

- Ta đang bảo vệ cái gì, khỏi ai và trong điều kiện nào.
- Plaintext, ciphertext, key, nonce, tag và AAD là gì.
- Vì sao đã mã hóa chưa đồng nghĩa với an toàn.
- Vì sao khóa quan trọng hơn tên thuật toán.
- Cách biến security property thành contract và test.

Dành cho người học Kotlin chưa có nền tảng security
Phạm vi: application-level encrypted local storage

1. Câu chuyện bắt đầu từ một file

Lan dùng TRACE để ghi nhớ:

- Balo được nhìn thấy lần cuối ở thư viện lúc 14:32.
- Chìa khóa nhà nằm trên bàn làm việc.
- Ảnh khuôn mặt của Minh được dùng làm reference.
- Mỗi lần vật xuất hiện đều kèm thời gian và vị trí.

Một ngày, một bản sao dữ liệu của TRACE xuất hiện trên laptop của người lạ:

```
trace.db
blobs/
backup/
```

Nguyên nhân có thể là cấu hình backup sai, thiết bị phục vụ debug, một bản export bị gửi nhầm, hoặc tương lai TRACE đồng bộ dữ liệu lên một storage khác. Ta chưa cần chọn nguyên nhân. Ta chỉ xét hậu quả: **attacker có các byte đã lưu.**

Hãy đặt câu hỏi đúng:

Có thể thiết kế dữ liệu đã lưu sao cho việc lấy được file chưa đủ để đọc hoặc sửa nội dung hay không?

Đó là bài toán của lecture.

"Local storage không đáng tin" có phải vấn đề tự nghĩ ra?

Nếu nói rằng SQLite vốn không đáng tin thì không chính xác. Android đã có hai lớp bảo vệ quan trọng:

- App sandbox ngăn ứng dụng bình thường khác đọc vùng dữ liệu riêng của TRACE.
- File-Based Encryption bảo vệ dữ liệu trên thiết bị, đặc biệt khi thiết bị đang khóa.

Vì vậy, nếu attacker chỉ là một app bình thường khác, cơ chế của Android đã giải quyết phần lớn bài toán.

Ta chỉ có bài toán mới khi đặt ra một attacker mạnh hơn nhưng vẫn cụ thể:

Attacker đã lấy được một bản sao offline của storage, nhưng chưa có khóa ứng dụng và chưa điều khiển tiến trình TRACE đang chạy.

Đây là **defense in depth**: thêm một lớp bảo vệ khi dữ liệu rời khỏi lớp bảo vệ ban đầu của Android. Nó không thay thế sandbox hay mã hóa của hệ điều hành.

Phạm vi trung thực

Thiết kế trong lecture có thể bảo vệ trước:

- Đọc một bản sao database hoặc file ảnh.
- Sửa byte trong ciphertext.
- Hoán đổi encrypted payload giữa hai context khác nhau.

Thiết kế không bảo vệ trước:

- Attacker điều khiển TRACE khi ứng dụng đang chạy.
- Attacker đọc plaintext trên màn hình hoặc trong RAM.
- Attacker có thể sử dụng khóa trong Android Keystore theo ý muốn.
- Xóa toàn bộ database.
- Khôi phục cả hệ thống về một snapshot cũ hợp lệ.

Security tốt không hứa "an toàn tuyệt đối". Nó nói chính xác lời hứa có hiệu lực trong điều kiện nào.

2. Ngôn ngữ của security

Trước khi chọn AES, ta cần một mô hình. Mỗi định nghĩa dưới đây trả lời một câu hỏi.

Asset - tài sản

Asset là thứ có giá trị cần được bảo vệ.

Với TRACE:

ảnh + embedding + vị trí + thời gian + ghi chú riêng tư

Không phải mọi dữ liệu đều phải bí mật. `schemaVersion = 1` có thể công khai, nhưng vị trí cuối cùng của một người thì nhạy cảm.

Câu hỏi: Nếu dữ liệu này bị đọc hoặc sửa, người dùng chịu thiệt hại gì?

Adversary - đối thủ

Adversary là kẻ ta đang thiết kế hệ thống để chống lại. Không nói "hacker" chung chung; phải liệt kê khả năng.

Attacker của bài này:

CÓ THỂ: đọc, sao chép, sửa, xóa, phát lại và hoán đổi byte đã lưu.
KHÔNG THỂ: lấy khóa, điều khiển app đang chạy hoặc đọc plaintext trong RAM.

Threat - mối đe dọa

Threat là một hành động của adversary làm hỏng một thuộc tính ta cần.

Ví dụ:

- Đọc ảnh từ file làm hỏng tính bí mật.
- Sửa tọa độ làm hỏng tính toàn vẹn.
- Đổi record của Alice sang Bob làm hỏng ràng buộc ngữ cảnh.

Trust boundary - ranh giới tin cậy

Trust boundary là nơi mức độ tin cậy thay đổi.

Trong mô hình đơn giản:

```

[Trusted]
TRACE process + secret key
      |
      | seal / open
      v
[Untrusted for secrecy and integrity]
database + file + backup

```

"Untrusted" không có nghĩa SQLite độc hại. Nó có nghĩa thiết kế không dựa vào SQLite để giữ bí mật hoặc chứng minh byte chưa bị sửa.

Security property - thuộc tính bảo mật

Ba thuộc tính của bài này:

- **Confidentiality:** không có key thì không suy ra plaintext.
- **Integrity:** dữ liệu bị sửa phải bị phát hiện.
- **Context binding:** dữ liệu của record A không dùng hợp lệ như record B.

Assumption và non-goal

Assumption là điều ta giả sử đúng để lời hứa security có hiệu lực. Ví dụ: key không bị lộ.

Non-goal là vấn đề ta chủ động không giải. Ví dụ: chống attacker chụp màn hình.

Nếu không viết assumption và non-goal, ta rất dễ quảng cáo một lời hứa lớn hơn thứ code thực sự cung cấp.

Threat model hoàn chỉnh trong một đoạn

TRACE bảo vệ ảnh, embedding và location record. Attacker biết source code và có thể đọc hoặc sửa toàn bộ storage offline. Attacker không có khóa và không kiểm soát tiến trình đang chạy. Hệ thống phải giữ bí mật plaintext, phát hiện sửa đổi và từ chối payload được mở dưới sai user, object hoặc record type. Availability, runtime compromise và rollback toàn hệ thống không thuộc phạm vi.

Đây là điểm khởi đầu. [AES-256-GCM](#) chỉ là một quyết định đến sau.

3. Dữ liệu không đi vào cipher dưới dạng object Kotlin

Mật mã xử lý **byte**, không hiểu **Object**, **JSON**, ảnh hay tọa độ.

Giả sử TRACE có object:

```

data class SecretSighting(
    val objectId: String,
    val latitude: Double,
    val longitude: Double,
    val capturedAt: Long,
)

```

Pipeline thật là:

```
SecretSighting
  |
  | serialize
  v
plaintext bytes
  |
  | encrypt
  v
ciphertext bytes
  |
  | encode để lưu hoặc truyền
  v
Base64 / BLOB
```

Byte

Byte là đơn vị dữ liệu gồm 8 bit. Cipher nhận và trả về mảng byte.

Encoding

Encoding là quy ước biểu diễn dữ liệu, không cung cấp bí mật.

Ví dụ UTF-8 biến text thành byte. Base64 biến byte thành ký tự để đặt trong JSON:

```
hello -> aGVsbG8=
```

Ai cũng có thể decode Base64. Vì vậy:

Base64 không phải encryption.

Serialization

Serialization biến object có cấu trúc thành byte. **Deserialization** làm ngược lại.

Một format lưu lâu dài phải xác định:

- Tên và kiểu của từng field.
- Thứ tự hoặc quy tắc canonical hóa nếu field được dùng làm AAD.
- Cách biểu diễn null, số và text.
- Phiên bản schema.

Ví dụ hai chuỗi này có cùng ý nghĩa JSON nhưng byte khác nhau:

```
{"objectId":"bag","version":1}
{"version":1,"objectId":"bag"}
```

Nếu một bên dùng chuỗi đầu làm AAD và bên kia tạo chuỗi thứ hai, xác thực sẽ thất bại. Vì vậy context cần một cách serialize xác định, không ghép chuỗi tùy ý.

Quyết định dữ liệu nào được mã hóa

Ta thường tách:

```
Public metadata:  envelopeVersion, algorithm, keyId, nonce
Secret payload:   image bytes, embedding, precise location, private note
Bound context:    userId, objectId, recordType, schemaVersion
```

Public metadata không cần bí mật. Nhưng nếu nó ảnh hưởng tới việc giải mã, code phải validate chặt chẽ.

4. Encryption từ nguyên lý đầu tiên

Các thành phần

- **Plaintext (P)**: dữ liệu gốc.
- **Key (K)**: bí mật điều khiển phép biến đổi.
- **Encryption**: biến plaintext thành ciphertext.
- **Ciphertext (C)**: dữ liệu đã mã hóa.
- **Decryption**: dùng key để khôi phục plaintext.
- **Cipher**: thuật toán thực hiện encryption và decryption.

Ta viết:

```
C = Encrypt(K, P)
P = Decrypt(K, C)
```

Correctness - tính đúng đắn

Một scheme phải giải mã lại đúng dữ liệu vừa mã hóa:

$$\text{Decrypt}(K, \text{Encrypt}(K, P)) = P$$

Đây mới là correctness, chưa phải security. Một thuật toán tự chế kiểu XOR với 123 có thể round-trip đúng nhưng vẫn bị phá rất dễ.

Symmetric encryption

Symmetric encryption sử dụng cùng một secret key cho seal và open. AES là một block cipher đối xứng.

Trong TRACE:

```
K = khóa vault
P = serialized sensitive payload
C = encrypted payload
```

Thuật toán được công khai

Ta giả sử attacker biết:

- Source code.
- Format envelope.

- TRACE dùng AES-GCM.
- Cách app serialize dữ liệu.

Chỉ **K** cần bí mật. Nếu hệ thống an toàn nhờ giấu tên thuật toán, đó là **security through obscurity**, không phải nền móng phù hợp cho mật mã.

Vì sao không chỉ nói "AES-256 rất mạnh"?

Một primitive mạnh vẫn có thể bị dùng sai:

- Hard-code key trong source.
- Lưu key cạnh database.
- Tái sử dụng nonce.
- Không xác thực ciphertext.
- Tin AAD do attacker gửi cùng payload.
- Log plaintext khi có lỗi.

Độ mạnh của key không sửa được những lỗi thiết kế này.

5. Encryption không tự động bảo vệ integrity

Giả sử ta có ciphertext của tọa độ:

```
8A 1F 40 72 ...
```

Attacker không đọc được plaintext, nhưng họ sửa một byte:

```
8A 1F 41 72 ...
```

Câu hỏi không phải chỉ là "attacker có đọc được không?". Câu hỏi thứ hai là:

TRACE có biết dữ liệu đã bị sửa không?

Integrity

Integrity là khả năng phát hiện thay đổi trái phép.

Nếu decrypt trả về dữ liệu rác nhưng app vẫn dùng, hệ thống đã thất bại. Fail an toàn phải là:

```
plaintext hợp lệ  
HOẶC  
authentication failure
```

Không tồn tại trạng thái "cố dùng plaintext gần đúng".

Authenticity

Trong bài này, **authenticity** có nghĩa payload hợp lệ phải được tạo bởi một bên có key tương ứng. Nó không nói người dùng nào đã nhấn nút; đó là bài toán identity khác.

Authentication tag

Authentication tag là giá trị kiểm tra được tạo trong quá trình seal. Khi open, cipher xác minh tag trước khi trả plaintext.

Nếu ciphertext, nonce hoặc AAD bị thay đổi, việc xác minh phải thất bại.

AEAD

Authenticated Encryption with Associated Data (AEAD) là một cơ chế cung cấp đồng thời:

confidentiality + integrity/authenticity + context authentication

AES-GCM là một AEAD scheme. Với bài này, "dùng AES" là mô tả thiếu; điều ta cần là một AEAD được dùng đúng contract.

Mô hình:

```
(ciphertext, tag) = Seal(K, nonce, plaintext, aad)
plaintext         = Open(K, nonce, ciphertext, tag, aad)
```

Open chỉ có hai kết quả hợp lệ:

```
Success(plaintext)
Failure(AuthenticationFailed)
```

6. Nonce - giá trị không bí mật nhưng không được lặp

Định nghĩa

Nonce là viết tắt của "number used once": một giá trị phải duy nhất cho mỗi lần seal dưới cùng một key.

Nonce thường:

- Không cần bí mật.
- Được lưu cạnh ciphertext.
- Phải được đưa đúng lại khi open.

Với AES-GCM, lựa chọn phổ biến là nonce 12 byte được sinh bằng bộ sinh số ngẫu nhiên mật mã.

Vì sao cùng plaintext cần ciphertext khác nhau?

Nếu hai lần lưu "University" luôn tạo cùng ciphertext, attacker chưa đọc được chữ nhưng có thể nhận ra hai record giống nhau. Randomized encryption làm giảm sự rò rỉ mẫu này.


```
Seal(K, "University", nonce_1) -> C1
Seal(K, "University", nonce_2) -> C2

nonce_1 != nonce_2
C1      != C2
```

Quy tắc sinh tử

Không bao giờ tái sử dụng cùng một cặp key + nonce với AES-GCM.

Nonce reuse có thể làm lộ quan hệ giữa plaintext và phá khả năng xác thực. Đây không phải lỗi "giảm một chút security"; nó có thể phá giả định cốt lõi của scheme.

Những cách làm sai

```
nonce = 000000000000
nonce = objectId bị cắt còn 12 byte
nonce = timestamp mà không chứng minh tính duy nhất
nonce = counter bị reset sau khi reinstall hoặc crash
```

Cách học phù hợp cho TRACE:

```
nonce = SecureRandom(12 bytes)
```

Sau đó test số lượng lớn để phát hiện lỗi triển khai. Test 10.000 nonce không "chứng minh toán học" chúng không bao giờ trùng; nó kiểm tra code có vô tình dùng hằng số, seed sai hoặc tái sử dụng buffer hay không.

Tag length

TRACE yêu cầu authentication tag 16 byte, tức 128 bit. Trong nhiều Java API, ciphertext và tag được trả chung bởi `doFinal`; storage format vẫn phải biết cách thư viện biểu diễn chúng.

7. AAD - khóa ciphertext vào đúng ngữ cảnh

Giả sử attacker không sửa ciphertext. Họ chỉ đổi chỗ hai envelope:

```
Envelope của object A -> record B
Envelope của object B -> record A
```

Mỗi envelope vẫn có tag hợp lệ. Nếu open không biết mình đang mở cho object nào, cả hai có thể giải mã thành công.

Associated Authenticated Data

AAD là dữ liệu:

- Không được mã hóa, nên không bí mật.

- Được authentication tag bảo vệ.
- Phải giống hệt khi seal và open.

Ví dụ context của TRACE:

```
userId      = user-42
objectId    = bag-17
recordType  = OBJECT_PHOTO
schemaVersion= 1
```

Ta serialize context theo một format ổn định rồi dùng làm AAD:

```
aad = EncodeContext(userId, objectId, recordType, schemaVersion)
```

Nếu caller yêu cầu mở envelope dưới `objectId = laptop-9`, AAD khác và xác thực thất bại.

Lỗi thiết kế tinh vi: tin AAD nằm trong chính envelope

Thiết kế yếu:

```
fun open(envelope: Envelope): ByteArray {
    return decrypt(
        ciphertext = envelope.ciphertext,
        aad = envelope.aad,
    )
}
```

Attacker có thể chuyển cả `ciphertext` + `aad` cùng nhau. Cipher chỉ chứng minh hai phần khớp với nhau; nó không biết caller mong đợi record nào.

Contract mạnh hơn:

```
fun open(
    envelope: Envelope,
    expectedContext: RecordContext,
): ByteArray
```

`expectedContext` phải đến từ quyết định của caller hoặc một nguồn tin cậy phù hợp, không được mù quáng lấy từ envelope đang bị kiểm tra.

Đây là khác biệt giữa:

```
"Payload tự kể nó là ai"
và
"Caller nói tôi đang chờ payload của ai"
```

AAD không giải quyết mọi hoán đổi

Nếu attacker kiểm soát cả envelope lẫn nguồn `expectedContext`, context binding không còn gốc tin cậy. Security luôn phụ thuộc trust boundary. AAD là công cụ ràng buộc; nó không tự tạo ra một nguồn identity đáng tin.

8. Encrypted envelope - format lưu trữ lâu dài

Cipher trả byte, nhưng sản phẩm cần một **envelope** có cấu trúc để lưu và mở lại sau này.

Định nghĩa

Encrypted envelope là container chứa ciphertext và metadata cần thiết để chọn cách mở nó.

Một thiết kế tối thiểu:

```
data class EncryptedEnvelope(  
    val envelopeVersion: Int,  
    val algorithm: String,  
    val keyId: String,  
    val nonce: ByteArray,  
    val cipherTextAndTag: ByteArray,  
)
```

Ý nghĩa:

- **envelopeVersion**: cách parse container.
- **algorithm**: policy cho phép thuật toán nào.
- **keyId**: chọn key, không phải secret key.
- **nonce**: nonce dùng khi seal.
- **cipherTextAndTag**: dữ liệu bí mật và authentication tag.

Version khác algorithm

Hai khái niệm dễ nhầm:

- **Envelope version** nói cách đọc format.
- **Algorithm** nói primitive/mode mật mã.

Ta có thể thay format mà vẫn dùng AES-GCM, hoặc đổi thuật toán trong cùng một khung versioning được thiết kế trước.

Validate trước khi dùng

Open phải từ chối có kiểm soát nếu:

- Version không hỗ trợ.
- Algorithm không nằm trong allowlist.
- Nonce sai độ dài.
- Base64/BLOB hỏng.
- **keyId** không tồn tại.
- Authentication tag không hợp lệ.

Không được tự động fallback sang một thuật toán yếu hơn chỉ để "cố mở" dữ liệu.

Parse error khác authentication failure

- **Parse error:** envelope không đúng cấu trúc để bắt đầu xác minh.
- **Authentication failure:** cấu trúc có thể đọc nhưng tag không hợp lệ cho key/context.

Bên trong code nên phân biệt để debug và xử lý. Nhưng API public không nên trả chi tiết nhạy cảm hoặc plaintext từng phần cho attacker.

9. Local key management - phần quan trọng hơn AES

Hãy tưởng tượng đặt tài liệu vào két sắt rồi dán chìa khóa lên cửa két. Két có thể rất tốt, nhưng hệ thống vẫn thất bại.

Điều tương tự xảy ra nếu:

```
trace.db
trace-aes-key.txt
```

được lưu cùng nơi.

Key generation

Key generation là tạo key bằng nguồn ngẫu nhiên mật mã. Key không phải password dễ nhớ, UUID, tên thiết bị hay chuỗi hard-code.

Với AES-256:

```
key length = 256 bit = 32 byte
```

Con số 256 chỉ có ý nghĩa nếu 32 byte thực sự ngẫu nhiên và được giữ bí mật.

Key storage

Key storage trả lời: key tồn tại ở đâu khi app tắt?

Trong Android production, hướng chuẩn là dùng Android Keystore để key có thể được giữ tách khỏi file ứng dụng và, trên thiết bị hỗ trợ, được bảo vệ bởi phần cứng.

Trong playground Kotlin không cần Android Studio, ta vẫn giữ kiến trúc đúng bằng interface:

```
interface KeyProvider {
    fun currentKeyId(): String
    fun getKey(keyId: String): SecretKey?
}
```

Dev implementation có thể lưu key trong một file dev được loại khỏi Git. Android implementation sau này thay bằng Keystore. Vault algorithm không cần biết key đến từ đâu.

Key persistence

Nếu constructor tự sinh key mới mỗi lần process khởi động:

```
Run 1: seal bằng K1  
Restart  
Run 2: open bằng K2 -> thất bại
```

Round-trip trong cùng một process vẫn pass, nhưng dữ liệu lưu lâu dài không dùng được. Đây là lý do test restart quan trọng.

Key ID

Key ID là tên công khai để chọn key, ví dụ `vault-key-v1`. Nó không phải key và không cần bí mật.

Envelope lưu `keyId`; `KeyProvider` tìm key tương ứng.

Key lifecycle

Vòng đời tối thiểu:

```
generate -> activate -> use -> retire -> destroy
```

- **Active key**: dùng để seal dữ liệu mới.
- **Retired key**: không seal mới nhưng còn dùng để open dữ liệu cũ.
- **Destroyed key**: không thể open dữ liệu cũ nữa.

Key rotation

Key rotation là chuyển sang key mới mà không làm mất dữ liệu cũ:

```
open envelope bằng oldKey  
seal plaintext bằng newKey  
ghi envelope mới một cách an toàn
```

Rotation không chỉ là đổi `keyId`. Dữ liệu phải thật sự được re-encrypt, hoặc hệ thống phải tiếp tục giữ old key để đọc bản cũ.

Key loss

Encryption đổi bài toán confidentiality thành bài toán availability:

Mất key thì dữ liệu hợp lệ cũng trở thành không thể đọc.

Vì vậy backup ciphertext mà không có chiến lược key tương ứng có thể tạo ra một bản backup không thể phục hồi. Đây là trade-off, không phải lỗi ngẫu nhiên.

10. Contract đúng cho TRACE Secure Vault

Ta tách ba trách nhiệm:

```
Serializer    object <-> plaintext bytes
KeyProvider   keyId <-> secret key
LocalVault    plaintext bytes <-> encrypted envelope
```

Contract cốt lõi:

```
interface LocalVault {
    fun seal(
        plaintext: ByteArray,
        context: RecordContext,
    ): EncryptedEnvelope

    fun open(
        envelope: EncryptedEnvelope,
        expectedContext: RecordContext,
    ): ByteArray
}
```

RecordContext có serialization ổn định:

```
data class RecordContext(
    val userId: String,
    val objectId: String,
    val recordType: String,
    val schemaVersion: Int,
)
```

Seal pipeline

1. Validate plaintext và context.
2. Lấy active key + keyId từ KeyProvider.
3. Sinh nonce mới 12 byte.
4. Serialize context thành AAD.
5. AES-GCM encrypt plaintext với key, nonce, AAD.
6. Tạo versioned envelope.
7. Chỉ trả envelope; không log key hoặc plaintext.

Open pipeline

1. Parse và validate envelope.
2. Dùng keyId để lấy đúng key.
3. Serialize expectedContext thành AAD.
4. AES-GCM verify + decrypt.
5. Chỉ trả plaintext sau khi authentication thành công.
6. Nếu thất bại, không trả plaintext từng phần và không thử "sửa hộ" dữ liệu.

Invariant

Invariant là điều phải luôn đúng.

Các invariant của vault:

```
Open(Seal(P, C), C) = P
Open(Seal(P, C1), C2) = Failure, nếu C1 != C2
Tamper(Seal(P, C)) = Failure
```

Invariant là cầu nối giữa lý thuyết security và test tự động.

11. Failure handling - thất bại cũng là một phần của security

Security code thường được đánh giá ở đường lỗi, không chỉ đường thành công.

Fail closed

Fail closed nghĩa là khi không chắc dữ liệu hợp lệ, hệ thống từ chối sử dụng.

```
Tag sai      -> từ chối
Key không có -> từ chối
Version lạ   -> từ chối
Nonce sai    -> từ chối
AAD sai      -> từ chối
```

Không có fallback như:

```
"GCM không mở được, thử coi ciphertext như plaintext"
```

Không rò rỉ qua log

Không log:

- Secret key.
- Plaintext.
- Full image bytes.
- Payload sau khi authentication thất bại.

Log có thể chứa:

```
operationId, envelopeVersion, keyId, errorCategory
```

miễn là các giá trị đó không chứa dữ liệu người dùng nhạy cảm.

Một error contract vừa đủ

```
sealed interface VaultError {
    data object UnsupportedEnvelope : VaultError
    data object KeyUnavailable : VaultError
    data object AuthenticationFailed : VaultError
    data object InvalidInput : VaultError
}
```

Không cần phân biệt cho caller rằng "ciphertext byte thứ 7 sai" hay "AAD sai". Chi tiết quá mức có thể tạo oracle và cũng không giúp app phục hồi.

Crash consistency

Nếu rotation hoặc ghi file bị dừng giữa chừng, không được để mất cả bản cũ lẫn bản mới. Pattern cơ bản:

```
write temporary -> flush -> atomic replace -> delete old when safe
```

Đây là phần giao nhau giữa storage engineering và security: dữ liệu bí mật nhưng không thể phục hồi vẫn là một hệ thống thất bại.

12. Test security property, không chỉ test API

Một test duy nhất kiểu `seal` rồi `open` chỉ chứng minh correctness. Bộ test phải đóng vai attacker.

Nhóm A - correctness

1. Đúng key + đúng context trả lại chính xác plaintext.
2. Payload rỗng, Unicode tiếng Việt và payload lớn hoạt động đúng.
3. Dữ liệu vẫn open được sau khi restart với cùng persistent key provider.

Nhóm B - confidentiality signals

4. Mã hóa cùng plaintext hai lần tạo nonce khác nhau.
5. Hai envelope không có ciphertext giống nhau trong test bình thường.
6. Ciphertext không chứa trực tiếp đoạn plaintext đã biết.

Test 6 không chứng minh mật mã an toàn; nó bắt lỗi nghiêm trọng như vô tình lưu plaintext vào field sai.

Nhóm C - integrity

7. Lật một bit ciphertext -> `AuthenticationFailed`.
8. Lật một bit authentication tag -> `AuthenticationFailed`.
9. Sửa nonce -> `AuthenticationFailed`.
10. Dùng wrong key -> `AuthenticationFailed`.

Nhóm D - context binding

11. Đổi `objectId` trong expected context -> thất bại.
12. Đổi `userId` -> thất bại.
13. Đổi `recordType` hoặc `schemaVersion` -> thất bại.
14. Hoán đổi envelope giữa hai record -> thất bại khi caller cung cấp context mong đợi.

Nhóm E - format và lifecycle

- 15. Envelope version lạ -> lỗi rõ ràng.
- 16. Algorithm không được hỗ trợ -> lỗi rõ ràng.
- 17. Nonce sai độ dài -> lỗi rõ ràng.
- 18. `keyId` không tồn tại -> `KeyUnavailable`.
- 19. Old envelope vẫn mở được bằng retired key trong giai đoạn rotation.

Nonce stress test

Sinh ít nhất 10.000 envelope với cùng key và kiểm tra không có nonce trùng trong mẫu test:

```
val seen = mutableSetOf<String>()
repeat(10_000) {
    val envelope = vault.seal(payload, context)
    check(seen.add(envelope.nonce.toBase64()))
}
```

Nhắc lại: test này bắt lỗi code; nó không thay thế lập luận xác suất hay yêu cầu dùng `SecureRandom`.

13. Đọc implementation hiện tại của TRACE như một security engineer

File trọng tâm:

```
playground/member4-vault/src/main/kotlin/
com/trace/playground/vault/VaultAlgorithm.kt
```

Khi đọc, không bắt đầu bằng việc sửa code. Hãy hỏi theo thứ tự:

Câu hỏi 1 - threat model có được contract thể hiện không?

`open` có nhận **expected context** từ caller không, hay tự tin AAD nằm trong payload?

Câu hỏi 2 - key có tồn tại qua restart không?

Nếu mỗi `VaultAlgorithm` tự `generateKey()`, dữ liệu từ process trước sẽ không mở được.

Câu hỏi 3 - nonce có mới cho mỗi lần seal không?

Kiểm tra nguồn randomness, độ dài và test lặp.

Câu hỏi 4 - tag failure được xử lý thế nào?

Code phải từ chối toàn bộ payload, không log plaintext và không trả dữ liệu một phần.

Câu hỏi 5 - envelope có thể tiến hóa không?

Cần version, algorithm policy và key ID đủ rõ để đọc dữ liệu cũ hoặc từ chối có kiểm soát.

Câu hỏi 6 - test đang chứng minh điều gì?

Gắn mỗi test với một property: correctness, nonce uniqueness, integrity, context binding hoặc key lifecycle.

Đây là cách đọc security code: đi từ lời hứa, qua trust boundary, tới invariant, rồi mới tới từng dòng implementation.

14. Bài tập duy nhất

Đề bài

Hoàn thiện module Secure Vault để bảo vệ một TRACE record trước attacker có thể đọc và sửa storage offline nhưng không có key.

API cần đạt

```
Seal(plaintext, context) -> versioned encrypted envelope  
Open(envelope, expectedContext) -> plaintext hoặc typed failure
```

Điều kiện nghiệm thu

- AES-256-GCM.
- Nonce ngẫu nhiên 12 byte, không tái sử dụng với cùng key.
- Authentication tag 16 byte.
- AAD được tạo từ context có serialization xác định.
- `open` nhận expected context, không chỉ tin context trong envelope.
- Key không hard-code, không log và tồn tại qua restart trong dev environment.
- Envelope có version và key ID.
- Có active/retired key tối thiểu để mô phỏng rotation.
- Tamper ciphertext, nonce, AAD hoặc wrong key đều thất bại.
- Ít nhất 10.000 nonce trong stress test không trùng.
- Không trả plaintext nếu authentication thất bại.

Không thuộc bài tập

- Login, JWT và authorization.
- HTTPS hoặc network encryption.
- Root detection.
- Mã hóa toàn bộ SQLite.
- UI Android.
- Cloud key management.
- Rollback protection hoàn chỉnh.

Cách trình bày khi bảo vệ

Không bắt đầu bằng "em dùng AES-256". Hãy trình bày:

1. Asset là gì?
2. Attacker làm được gì?
3. Trust boundary nằm ở đâu?
4. Security properties là gì?
5. Contract nào thể hiện các property đó?
6. Test nào chứng minh implementation tuân thủ contract?
7. Non-goal và residual risk còn lại là gì?

Nếu trả lời được bảy câu này, bạn đã giải một bài toán security. AES-GCM chỉ là công cụ được chọn để hiện thực lời hứa đó.

15. Bản đồ tư duy cuối bài

Hãy giữ chuỗi suy luận sau:

```

File có thể bị lấy hoặc sửa
|
v
Xác định asset, attacker, assumption, non-goal
|
v
Đặt mục tiêu: confidentiality + integrity + context binding
|
v
Serialize object thành plaintext bytes và context thành AAD
|
v
AEAD seal bằng key bí mật + nonce duy nhất
|
v
Lưu versioned envelope, không lưu key cạnh ciphertext
|
v
Open bằng expected context, fail closed nếu xác thực sai
|
v
Test như attacker: sửa, đổi, dùng sai key, restart, rotate

```

Sáu câu định nghĩa cần nhớ

1. **Encryption** bảo vệ ý nghĩa của plaintext khỏi người không có key.
2. **Integrity** yêu cầu mọi sửa đổi trái phép phải bị phát hiện.
3. **AEAD** cung cấp encryption và authentication trong cùng một scheme.
4. **Nonce** không cần bí mật nhưng không được lặp với cùng key.
5. **AAD** ràng buộc ciphertext với context mà không mã hóa context đó.
6. **Key management** quyết định ai thực sự có thể sử dụng cơ chế mật mã và dữ liệu có còn đọc được về sau hay không.

Một câu kết

Security không bắt đầu từ tên thuật toán. Nó bắt đầu từ một lời hứa chính xác về điều attacker không thể làm, rồi kết thúc bằng test cho thấy code giữ được lời hứa đó.

Tài liệu tham khảo chính

1. Android Developers, *Security best practices*:
<https://developer.android.com/privacy-and-security/security-best-practices>
2. Android Open Source Project, *Encryption*:
<https://source.android.com/docs/security/features/encryption>
3. Android Developers, *Android Keystore system*:
<https://developer.android.com/privacy-and-security/keystore>
4. NIST SP 800-38D, *Galois/Counter Mode (GCM) and GMAC*:
<https://csrc.nist.gov/pubs/sp/800/38/d/final>
5. Oracle, *Java Cryptography Architecture Reference Guide*:
<https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html>

TRACE Secure Vault - Lecture 01

Phạm vi: application-level encrypted local storage

Đối tượng: người học Kotlin chưa có nền tảng security